

Лабораторная работа №4

Написание Unit-тестов

Теоретическая часть и методика тестирования

Ручное тестирование:

— зная алгоритм нахождения периметра и площади фигуры, определяем наборы входных данных, которые будут переданы на вход программе;

— зная входные данные, можно вручную просчитать, какой ответ должна дать программа;

— запускаем программу и передаем ей на вход исходные данные;

— получаем от нее ответ и сравниваем с тем, который должен быть получен. Если они совпадают — хорошо, идём к следующему набору данных, если нет, сообщаем об ошибке, фиксируем в протоколе тестирования.

Формат и требования к Unit-тестам:

- Тесты должны быть написаны в виде тестового класса;
- Класс должен быть унаследован от базового класса `unittest.TestCase`;
- Имена всех тестовых функций должны начинаться с префикса `test`;
- Внутри функций вызываются операторы утверждения (`assertX`), проверяющие соответствие полученных результатов эталонным;
- Запуск тестов осуществляется командой: `python -m unittest <имя_файла>.py`.

План и результаты тестирования

1. Цели и задачи тестирования

Цель: Проведение автоматизированного модульного тестирования библиотеки геометрических функций для подтверждения корректности математических расчетов площади и периметра фигур.

Задачи:

1. Разработать модульные тесты с использованием фреймворка `unittest` для всех четырех модулей библиотеки: `circle.py`, `rectangle.py`, `square.py`, `triangle.py`.
2. Проверить правильность расчетов функций при стандартных положительных значениях параметров.
3. Проверить граничные условия (передача нулевых значений параметров).

2. Описание тестируемого продукта

Тестируемый продукт представляет собой библиотеку на языке Python, состоящую из четырех модулей и реализующую математические формулы евклидовой геометрии для расчета параметров плоских геометрических фигур (круг, прямоугольник, квадрат, треугольник).

Требования к продукту:

- Функция `area` должна возвращать площадь фигуры, рассчитанную по корректной математической

формуле;

- Функция `perimeter` должна возвращать суммарную длину всех сторон фигуры;

3. Область тестирования

В область тестирования включены модули и функции:

- Модуль `circle.py`: функции `area(r)` и `perimeter(r)`;
- Модуль `rectangle.py`: функции `area(a, b)` и `perimeter(a, b)`;
- Модуль `square.py`: функции `area(a)` и `perimeter(a)`;
- Модуль `triangle.py`: функции `area(a, h)` и `perimeter(a, b, c)`.

4. Стратегия тестирования

- Уровень тестирования: Модульное (Unit-тестирование).
- Техники тест-дизайна: Анализ классов эквивалентности (стандартные целые положительные значения) и анализ граничных значений (нулевые аргументы: $r = 0$, $a = 0$, $b = 0$, $h = 0$).
- Инструменты: Стандартный модуль Python `unittest`.

5. Критерии приемки

- Все разработанные тест-кейсы должны быть исполнены и задокументированы;
- Тест считается успешно пройденным (PASSED), если возвращаемое функцией значение строго совпадает с теоретически верным результатом;
- При несовпадении тест помечается статусом FAILED, что фиксирует наличие программного дефекта в соответствующем модуле.

6. Ожидаемые результаты

- Подготовлено 24 автоматизированных модульных теста (по 6 тестов на фигуру: 3 сценария для площади и 3 сценария для периметра);
- Формирование протокола тестирования с подробным описанием причин падения (`AssertionError`);
- Локализация дефектов в исходных файлах функций.

7. Актуальные результаты тестирования

Тестирование проводилось запуском каждого модуля через `python -m unittest`. Ниже представлена сводка выполнения тестов:

```

zsh: no such file or directory: venv/bin/python
kirill@MacBook-Air-Kirill-4 lab 4 % .venv/bin/python -m unittest -v circle rectangle square triangle.
test_area_0 (circle.CircleTestClass.test_area_0) ... ok
test_area_1 (circle.CircleTestClass.test_area_1) ... ok
test_area_2 (circle.CircleTestClass.test_area_2) ... ok
test_perimeter_0 (circle.CircleTestClass.test_perimeter_0) ... ok
test_perimeter_1 (circle.CircleTestClass.test_perimeter_1) ... ok
test_perimeter_2 (circle.CircleTestClass.test_perimeter_2) ... ok
test_area_1 (rectangle.RectangularTestClass.test_area_1) ... ok
test_area_2 (rectangle.RectangularTestClass.test_area_2) ... ok
test_area_zero (rectangle.RectangularTestClass.test_area_zero) ... ok
test_perimeter_1 (rectangle.RectangularTestClass.test_perimeter_1) ... FAIL
test_perimeter_2 (rectangle.RectangularTestClass.test_perimeter_2) ... FAIL
test_perimeter_zero (rectangle.RectangularTestClass.test_perimeter_zero) ... FAIL
test_area_0 (square.SquareTestClass.test_area_0) ... ok
test_area_1 (square.SquareTestClass.test_area_1) ... ok
test_area_2 (square.SquareTestClass.test_area_2) ... ok
test_perimeter_0 (square.SquareTestClass.test_perimeter_0) ... ok
test_perimeter_1 (square.SquareTestClass.test_perimeter_1) ... ok
test_perimeter_2 (square.SquareTestClass.test_perimeter_2) ... ok
(unittest.loader._FailedTest) ... ERROR

=====
ERROR: (unittest.loader._FailedTest)
=====
AttributeError: module 'triangle' has no attribute ''

=====
FAIL: test_perimeter_1 (rectangle.RectangularTestClass.test_perimeter_1)
=====
Traceback (most recent call last):
  File "/Users/kirill/Desktop/wcpno/lab 4/rectangle.py", line 13, in test_perimeter_1
    self.assertEqual(res, 30)
    ~~~~~^~~~~~
AssertionError: 15 != 30

=====
FAIL: test_perimeter_2 (rectangle.RectangularTestClass.test_perimeter_2)
=====
Traceback (most recent call last):
  File "/Users/kirill/Desktop/wcpno/lab 4/rectangle.py", line 23, in test_perimeter_2
    self.assertEqual(res, 14)

```

```

=====
ERROR: (unittest.loader._FailedTest.)
=====
AttributeError: module 'triangle' has no attribute ''
=====
FAIL: test_perimeter_1 (rectangle.RectangularTestClass.test_perimeter_1)
=====
Traceback (most recent call last):
  File "/Users/kirill/Desktop/испно/lab 4/rectangle.py", line 13, in test_perimeter_1
    self.assertEqual(res, 30)
    ~~~~~
AssertionError: 15 != 30
=====
FAIL: test_perimeter_2 (rectangle.RectangularTestClass.test_perimeter_2)
=====
Traceback (most recent call last):
  File "/Users/kirill/Desktop/испно/lab 4/rectangle.py", line 23, in test_perimeter_2
    self.assertEqual(res, 14)
    ~~~~~
AssertionError: 7 != 14
=====
FAIL: test_perimeter_zero (rectangle.RectangularTestClass.test_perimeter_zero)
=====
Traceback (most recent call last):
  File "/Users/kirill/Desktop/испно/lab 4/rectangle.py", line 18, in test_perimeter_zero
    self.assertEqual(res, 14)
    ~~~~~
AssertionError: 7 != 14
=====

Ran 19 tests in 0.001s

FAILED (failures=3, errors=1)
○ kirill@MacBook-Air-Kirill-4 lab 4 %

```

Модуль	Всего тестов	Успешно (ОК)	Провалено (FAILED)
circle.py (CircleTestClass)	6	6	0
square.py (SquareTestClass)	6	6	0
triangle.py (TriangleTestClass)	6	6	0
rectangle.py (RectangularTestClass)	6	3	3

Детализация проваленных тестов модуля rectangle.py:

Тест-кейс	Ожидалось	Фактический результат	Статус
test_perimeter_1 (perimeter(10, 5))	30	15	FAILED
test_perimeter_zero (perimeter(7, 0))	14	7	FAILED

test_perimeter_2
(perimeter(5, 2))

14

7

FAILED

8. Выводы из результатов тестирования

1. Разработанный набор unit-тестов успешно выполняет автоматизированную проверку функциональности геометрических модулей.
2. Модули circle.py, square.py и triangle.py прошли все проверки без ошибок (100% успешных тестов) и полностью соответствуют геометрическим формулам.
3. В модуле rectangle.py выявлена программная ошибка: в теле функции perimeter(a, b) записано выражение `return a + b` (полупериметр) вместо `return 2 * (a + b)`.